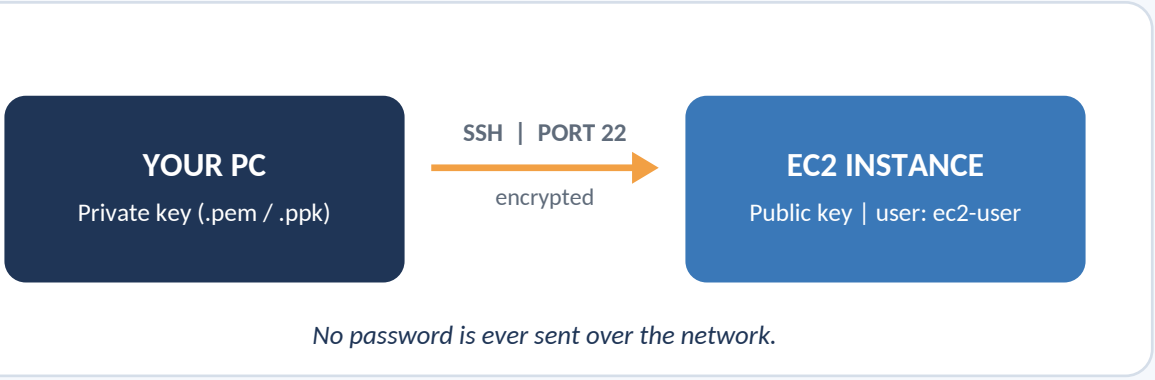
AMAZON WEB SERVICES | EC2 | TECHNICAL GUIDE

Connect to AWS EC2 with an SSH Key Pair

A step-by-step guide using Windows CMD, Git Bash, MobaXterm and PuTTY



No password is ever sent over the network.

FOUR WAYS TO LOGIN

CMD
Key format: .pem

Git Bash
Key format: .pem

MobaXterm
Key format: .pem

PuTTY
Key format: .ppk

4	20	Port 22	17
SSH CLIENTS	SCREENSHOTS	SECURE SHELL	GUIDED STEPS

DOCUMENT	Step-by-step guide
PLATFORM	AWS EC2 (Amazon Linux 2023)
REGION	Europe (Stockholm), eu-north-1
AUDIENCE	Beginners / Cloud learners

Overview

When you launch an EC2 server with a key pair, you log in with a private key file instead of a password. This is the standard, secure way to reach a Linux server in AWS. This guide shows how to create the key pair and then connect to the same server using four different Windows SSH clients, exactly as captured in your screenshots.

How key-pair login works



AWS keeps only the public half on the server. You download the private half once, when you create the key pair. Only a computer holding the matching private key can log in, so no password is ever sent over the network.

The four methods at a glance

Client	Key format	Best for
Windows CMD	.pem	Fastest option. OpenSSH is built into Windows 10 and 11, so nothing to install.
Git Bash	.pem	A Linux-style shell on Windows. Commands such as chmod work.
MobaXterm	.pem	Saved sessions, tabs and a built-in file browser (SFTP) for uploading and downloading files.
PuTTY	.ppk	The classic graphical SSH client. Requires the key in PuTTY's own .ppk format.

Before you start

- An AWS account and a Windows 10 or 11 computer (macOS and Linux users can use the same ssh command in Terminal).
- Internet access and permission to launch an EC2 instance.
- For methods 3 and 4, install MobaXterm and PuTTY (PuTTY includes the PuTTYgen tool).

> **NOTE**
> Your screenshots use the Europe (Stockholm), eu-north-1 region and the user name `ec2-user`, which is the default for Amazon Linux. Other operating systems use different names (for example `ubuntu` for Ubuntu).

Part 1 - Launch an Instance with a Key Pair

The first five steps create a server named **my-wit-key-connect** and generate the key pair **my-all-key**.

1 Name the instance and pick Amazon Linux

In the EC2 console choose **Launch instance**. Under **Name and tags** enter a name (here **my-wit-key-connect**). In **Application and OS Images** keep **Amazon Linux** selected on the Quick Start tab.

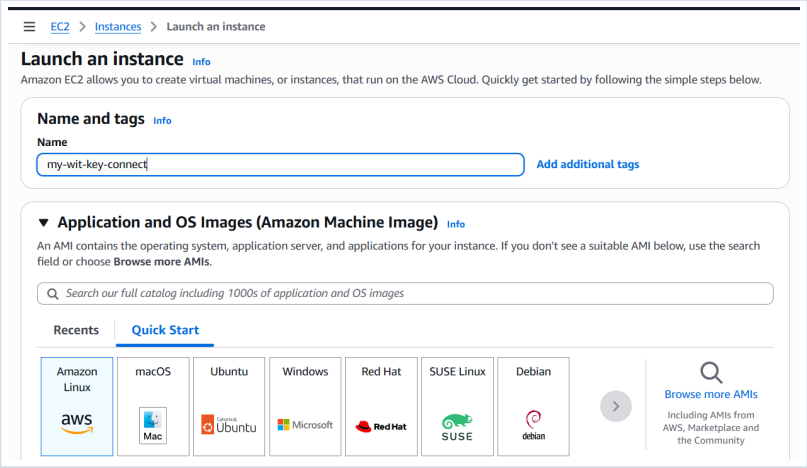


Figure 1 - Naming the instance and choosing the Amazon Linux quick start.

2 Confirm the AMI and instance type

The AMI is **Amazon Linux 2023 (kernel 6.18)**, marked Free tier eligible, 64-bit (x86), with default user name **ec2-user**. Choose the **t3.micro** instance type (2 vCPU, 1 GiB memory, Free Tier eligible).

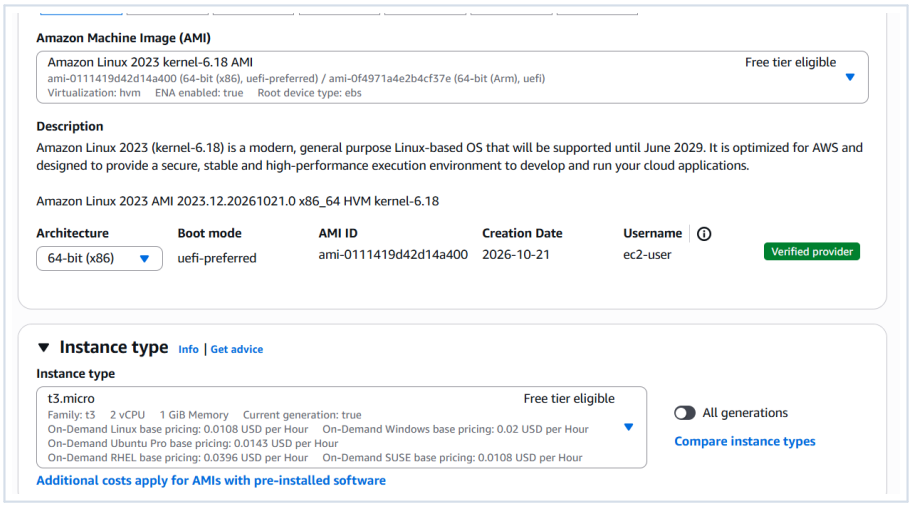


Figure 2 - AMI details (note the Username field) and the t3.micro instance type.

3 Create the key pair

Scroll to **Key pair (login)**, click **Create new key pair** and fill in the dialog:

- **Key pair name: my-all-key**. Use a name you will recognise later.
- **Key pair type: RSA** (widely supported). ED25519 is a newer, shorter alternative.
- **Private key file format:** choose **.pem** for OpenSSH clients (CMD, Git Bash, MobaXterm, macOS, Linux) or **.ppk** for PuTTY.
- Click **Create key pair**. The browser downloads the private key file immediately.

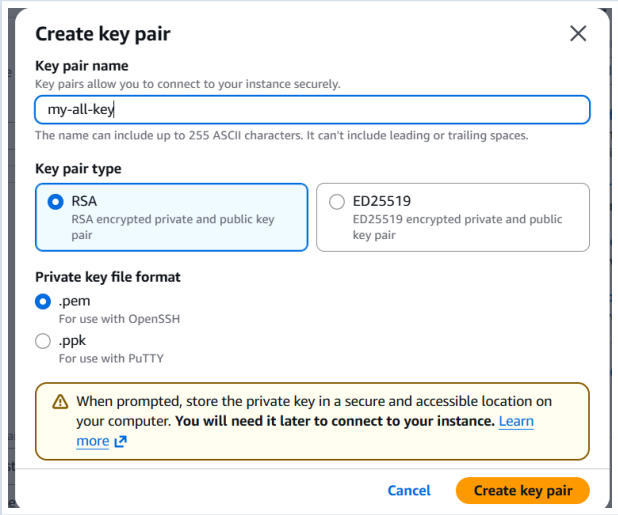


Figure 3 - The Create key pair dialog with RSA and .pem selected.

IMPORTANT

AWS shows the private key **only once**. If you lose the file you cannot download it again and must create a new key pair. Keep it in a safe folder and never share it.

4 Check storage and network

Keep the default **8 GiB gp3** root volume and **File systems: None**. In **Network settings** make sure **Allow SSH traffic** is ticked, because SSH uses port 22. For better security set its source to **My IP** instead of Anywhere.

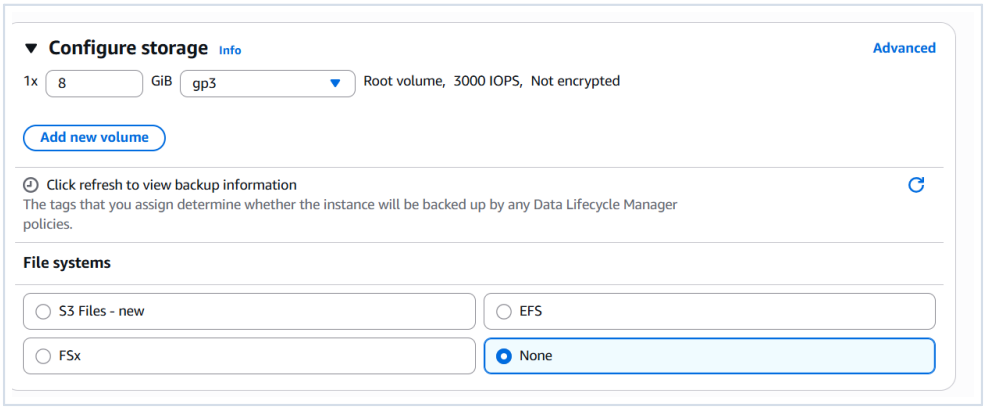


Figure 4 - Default storage configuration.

5 Review the summary and launch

The Summary shows 1 instance, Amazon Linux 2023, t3.micro, a new security group and 8 GiB of storage. Click **Launch instance**, then wait until the instance state is **Running**.

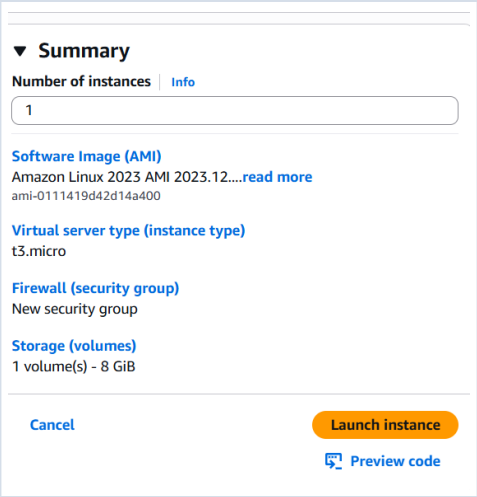


Figure 5 - Summary panel with the Launch instance button.

Part 2 - Find Your Connection Details

6 Open the Connect page and choose the SSH client tab

Go to **EC2 > Instances**, select your instance, click **Connect** and open the **In SSH client** tab. This page gives you everything every method below needs.

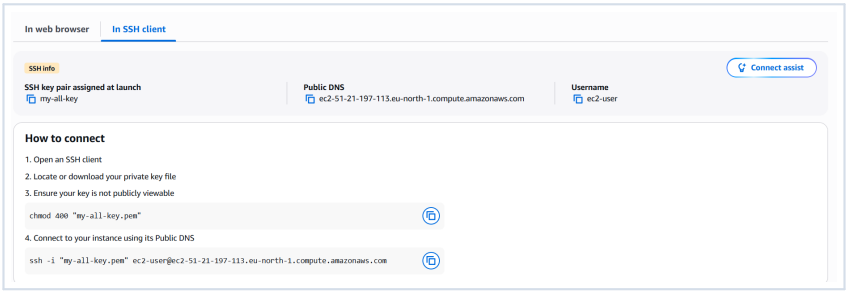


Figure 6 - The SSH client tab: key pair name, Public DNS, user name and ready-made commands.

Field on the page	Value in your example	Used for
SSH key pair assigned at launch	`my-all-key`	Tells you which key file to use
Public DNS	`ec2-51-21-197-113.eu-north-1.compute.amazonaws.com`	The address to connect to (the public IPv4 `51.21.197.113` also works)
Username	`ec2-user`	The Linux account to log in as

The two commands shown by AWS

```
chmod 400 "my-all-key.pem"
ssh -i "my-all-key.pem" ec2-user@ec2-51-21-197-113.eu-north-1.compute.amazonaws.com
```

Part	What it does
`chmod 400`	Makes the key file readable only by you. SSH refuses keys that other users can read. Run it on Git Bash, macOS and Linux (CMD does not have chmod).
`ssh`	The Secure Shell client program that opens an encrypted remote session.
`-i "my-all-key.pem"`	**i**dentity file: tells ssh which private key to present to the server.
`ec2-user@<Public-DNS>`	**user@host**: log in as **ec2-user** on the server with that Public DNS name.

> **TIP**
> The Public IP and Public DNS **change every time you stop and start** the instance (unless you attach an Elastic IP). Always copy the fresh values from this page before connecting.

Part 3 - Method 1: Windows Command Prompt (CMD)

Windows 10 and 11 include the OpenSSH client, so the plain Command Prompt is enough.

7 Open CMD and go to the folder that holds the key

Press **Win + R**, type `cmd` and press Enter. Browsers save downloads in the **Downloads** folder, so move there:

```
C:\Users\PCC> cd downloads
```

cd means *change directory*. The prompt becomes `C:\Users\PCC\Downloads>`. Now paste the ssh command you copied from AWS (right-click pastes in CMD):



Figure 7 - Command Prompt: change to Downloads and type the ssh command.

8 Press Enter and accept the host fingerprint

The first time you connect, SSH does not know the server yet and asks you to trust it:

- **The authenticity of host … can't be established** - normal for a new server. It shows the server's ED25519 fingerprint.
- Type `yes` and press Enter. SSH stores the host in your `known_hosts` file so it will not ask again.
- The **Amazon Linux 2023** logo appears and the prompt changes to `[ec2-user@ip-172-31-36-209 ~]$`. You are now inside the server.

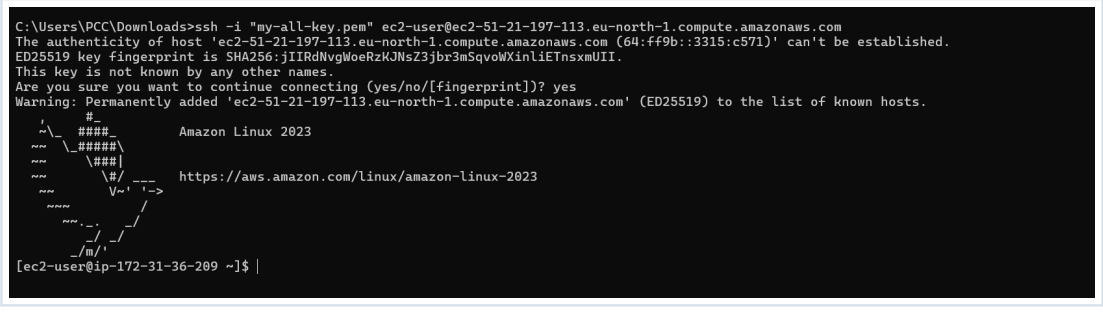


Figure 8 - First connection: type yes, then the Amazon Linux prompt appears.

CMD TIP

If you see **WARNING: UNPROTECTED PRIVATE KEY FILE**, Windows thinks other accounts can read the key. Fix it with the two commands in the Troubleshooting section, then try again.

Part 4 - Method 2: Git Bash

Git Bash comes with Git for Windows and gives you a Linux-like terminal, so the AWS commands work exactly as printed.

9 Open Git Bash, go to Downloads and connect

```
$ cd downloads
$ chmod 400 "my-all-key.pem"
$ ssh -i "my-all-key.pem" ec2-user@ec2-51-21-197-113.eu-north-1.compute.amazonaws.com
```

- Open Git Bash from the Start menu (or right-click a folder and choose **Open Git Bash here**).
- **cd downloads** moves to the key's folder. The coloured prompt in your screenshot also shows **(main)**; that is only Git Bash displaying a Git branch and can be ignored.
- Paste the **ssh** command and press Enter. On later logins no *yes* prompt appears for the same server.
- The line **connection is not using a post-quantum key exchange algorithm** is an informational warning from a newer SSH client. The session is still encrypted and works normally.



Figure 9 - Git Bash: the same ssh command, ending at the ec2-user prompt.

Part 5 - Method 3: MobaXterm

MobaXterm saves your sessions, opens multiple tabs and adds a file browser on the left so you can drag and drop files to the server.

10 Open MobaXterm and start a new SSH session

Launch MobaXterm. On the home screen click **Session** in the top-left toolbar. Your earlier sessions appear in the **User sessions** list on the left and under **Recent sessions**.

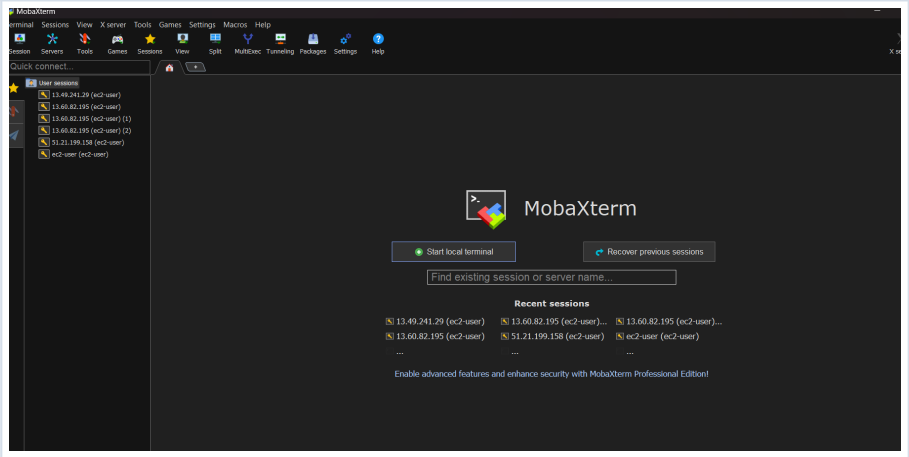


Figure 10 - MobaXterm home screen with the Session button and saved sessions.

11 Fill in the SSH session settings

In the Session settings window choose the **SSH** icon, then enter:

- **Remote host:** the instance's Public IPv4 address, `51.21.197.113`.
- **Username:** `ec2-user` (tick the box next to it if your version shows one).
- **Port:** `22`.
- Open the **Advanced SSH settings** tab, tick **Use private key** and click the file icon to browse to `my-all-key.pem` in your Downloads folder.
- Click **OK**.

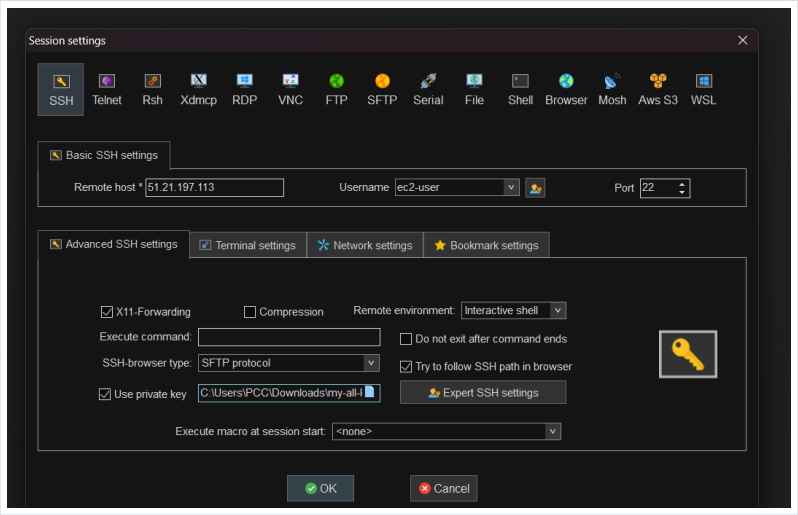


Figure 11 - SSH session settings with host, user name, port and private key.

12 You are connected

MobaXterm opens a new tab and prints a summary box. A green dot beside the session shows it is live.

- **Direct SSH** and **SSH-browser** show a green tick. The SSH-browser is the SFTP panel for transferring files.
- **X11-forwarding** shows a red cross because it is switched off on the server. You do not need it for normal command-line work; you can untick it in the session settings.
- The prompt `[ec2-user@ip-172-31-36-209 ~]$` confirms you are logged in.

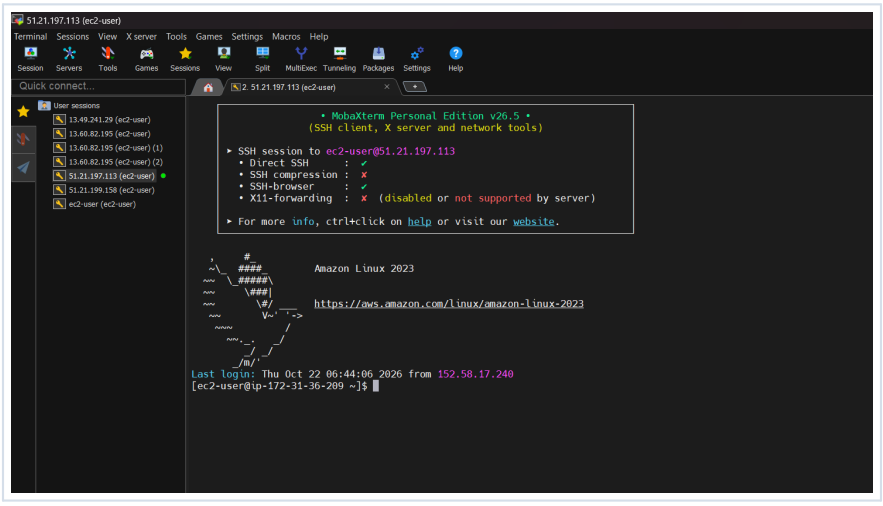


Figure 12 - A connected MobaXterm session with the Amazon Linux banner.

> **TIP**
> Next time just double-click the saved session in the left panel. If the instance was stopped and started, edit the session and update the Remote host with the new public IP.

Part 6 - Method 4: PuTTY

PuTTY cannot read a **.pem** file. It needs a **.ppk** key. You can either choose **.ppk** when creating the key pair (Step 3) or convert an existing .pem with **PuTTYgen**.

13 Get a .ppk key (convert with PuTTYgen)

Search the Start menu for **puttygen** and open **PuTTY Key Generator** (it is installed together with PuTTY).

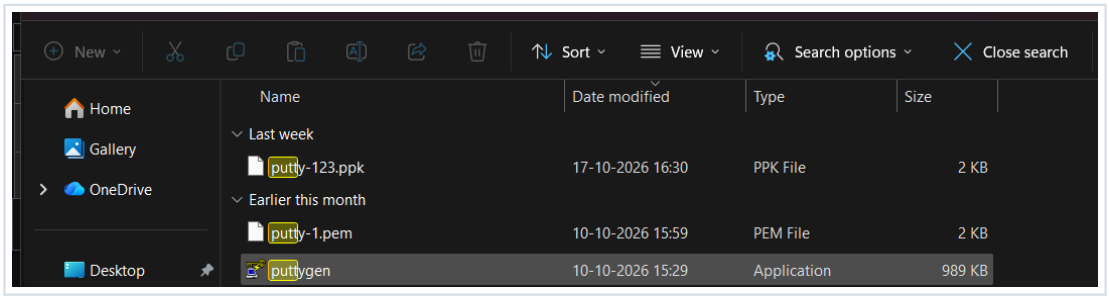


Figure 13 - Searching for puttygen in File Explorer / Start menu.

In PuTTYgen choose **File > Load private key**. The dialog only lists *.ppk files by default, so change the file-type drop-down to **All Files (*.*)** to see your .pem file, select it and click **Open**. Then click **Save private key** (accept the no-passphrase warning if you want passwordless login) and save it, for example as **my-all-key.ppk**.

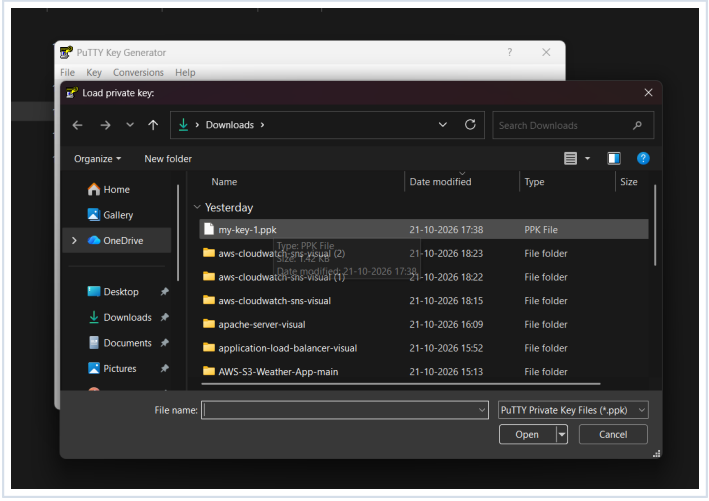


Figure 14 - PuTTYgen Load private key dialog (set the file type to All Files to see .pem).

SKIP THIS STEP
If you already created the key pair as **.ppk** in AWS, you already have the right file (for example **my-dd-pair.ppk** in your screenshots).

14 Enter the host in PuTTY

Open **PuTTY**. On the Session page enter the Public IPv4 address `51.21.197.113` as the **Host Name**, keep **Port 22** and make sure the connection type is **SSH**.

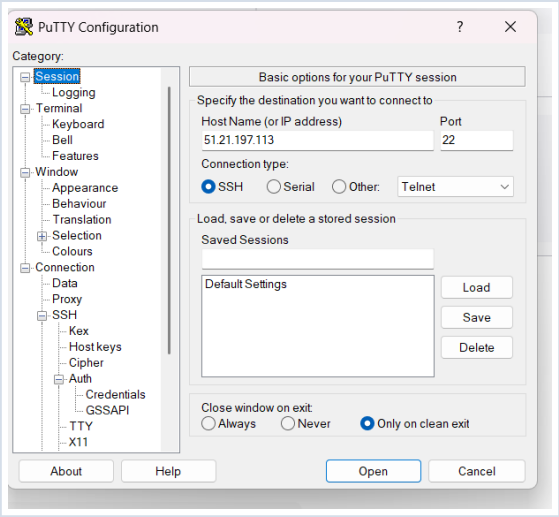


Figure 15 - PuTTY Configuration, Session page.

15 Load the private key under Credentials

In the **Category** tree on the left expand **Connection > SSH > Auth** and click **Credentials**. Under **Private key file for authentication** click **Browse...**.

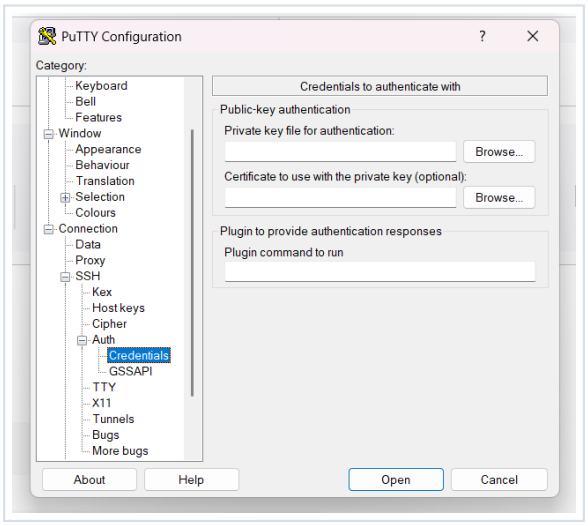


Figure 16 - Credentials page with an empty private key field.

In the file dialog (filtered to PuTTY Private Key Files, *.ppk) choose your .ppk key and click **Open**.

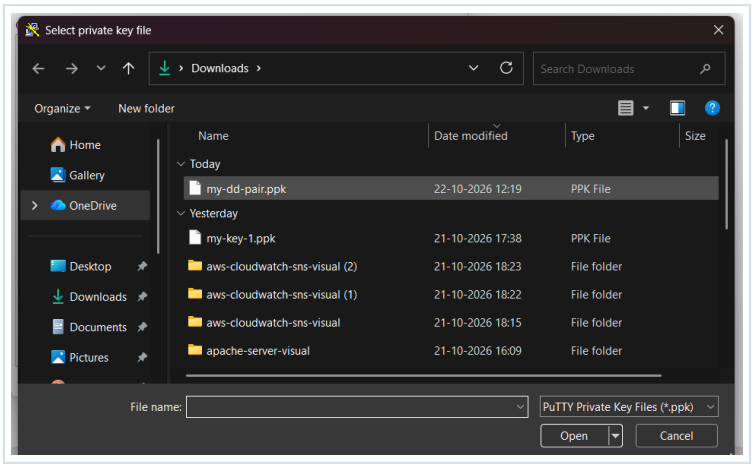


Figure 17 - Selecting the .ppk file from Downloads.

The full path now appears in the field, for example `C:\Users\PCC\Downloads\my-dd-pair.ppk`.

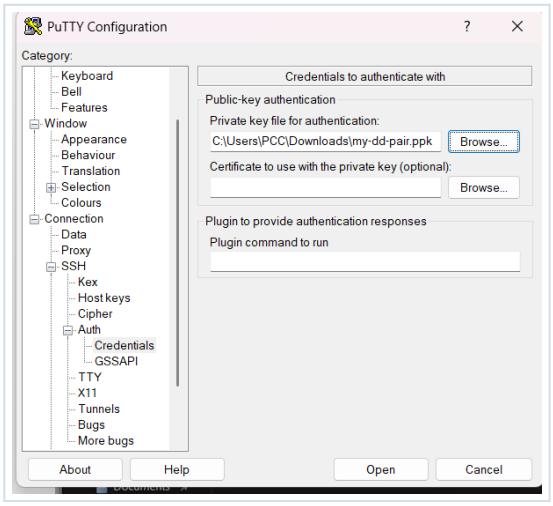


Figure 18 - Private key path filled in.

16 Click Open and accept the host key

Click **Open** at the bottom. On the first connection PuTTY shows a **Security Alert** because the server's host key is not cached yet. Compare the fingerprint if you wish and click **Accept** to trust it permanently (**Connect Once** trusts it for this session only; **Cancel** abandons the connection).

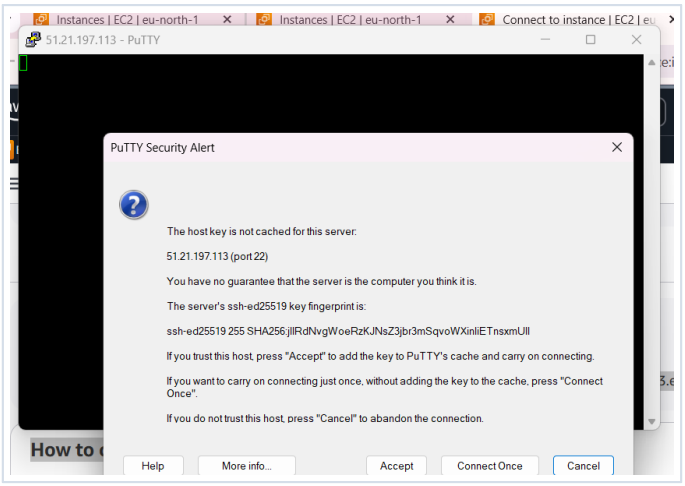


Figure 19 - PuTTY Security Alert on the first connection.

17 Log in as ec2-user

A black terminal opens with **login as:** and a blank cursor. Type `ec2-user` and press Enter. PuTTY reports **Authenticating with public key**, shows the Amazon Linux banner and the prompt `[ec2-user@ip-172-31-36-209 ~]$`. You are connected.

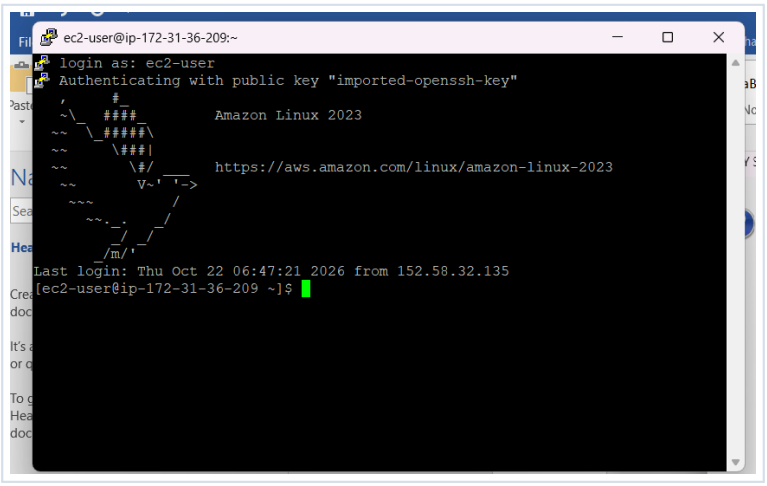


Figure 20 - Connected with PuTTY using the public key.

> **TIME SAVER**
> Skip the **login as:** prompt: go to **Connection > Data** and enter **ec2-user** under **Auto-login username**. Then return to **Session**, type a name under **Saved Sessions** and click **Save**. Next time double-click the saved session.

Part 7 - Verify the Connection

Whichever client you used, run these commands to confirm you are on the right server and understand where you are.

```
[ec2-user@ip-172-31-36-209 ~]$ whoami
ec2-user
[ec2-user@ip-172-31-36-209 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-36-209 ~]$ cat /etc/os-release
NAME="Amazon Linux" ... VERSION="2023"
[ec2-user@ip-172-31-36-209 ~]$ exit
logout
```

Command	What it does
`whoami`	Prints the user you are logged in as. Expect ec2-user.
`hostname`	Shows the server's name, which matches the prompt (for example ip-172-31-36-209).
`pwd`	**p**rint **w**orking **d**irectory: your current folder. A new login starts in /home/ec2-user (shown as ~).
`cat /etc/os-release`	Displays operating system details to confirm Amazon Linux 2023.
`sudo yum update -y`	Optional: updates all installed software. sudo runs it with administrator rights, -y auto-confirms.
`exit`	Closes the SSH session and returns you to your own computer.

Part 8 - Quick Reference

Client	Key	How to connect
CMD	.pem	`cd downloads`, then `ssh -i "my-all-key.pem" ec2-user@<Public-DNS>`
Git Bash	.pem	Same as CMD; run `chmod 400 "my-all-key.pem"` first.
MobaXterm	.pem	Session > SSH > Remote host, ec2-user, port 22 > Advanced SSH settings > Use private key > OK.
PuTTY	.ppk	Host Name = public IP > Connection > SSH > Auth > Credentials > Browse .ppk > Open > Accept > login as: ec2-user.

Part 9 - Protect Your Key

- Treat the private key like a password. Anyone who has the file can log in to your server.
- Never share or upload it to GitHub, chat groups, e-mail or screenshots.
- Move it out of Downloads into a protected folder once you are done, and keep a secure backup.
- Restrict permissions: `chmod 400` on Linux, macOS and Git Bash.
- Limit SSH access: set the security group rule for port 22 to **My IP** rather than 0.0.0.0/0.
- Use a separate key per project, delete unused key pairs, and if a key is exposed create a new pair and replace it.
- Stop or terminate the instance when you finish. Stopping keeps the disk (small cost) but changes the public IP on restart; terminating deletes the server and stops all charges.

Part 10 - Troubleshooting

Message or symptom	Likely cause	Fix
`Permission denied (publickey)`	Wrong user name, or not the key pair assigned at launch	Use ec2-user (Amazon Linux). Check the key pair name on the SSH client tab matches your file.
`UNPROTECTED PRIVATE KEY FILE`	Other users can read the key file	Git Bash/macOS/Linux: `chmod 400 key.pem`. Windows CMD: see the commands below.
`Connection timed out`	Port 22 blocked, instance not running, wrong or old IP, or your own IP changed	Check the security group allows SSH from your IP, the instance is Running, and copy the current Public IP/DNS.
`REMOTE HOST IDENTIFICATION HAS CHANGED`	A new server now uses an IP address you connected to before	Run `ssh-keygen -R <host>` to remove the old entry, then reconnect.
PuTTY: **Unable to use key file** or **No supported authentication methods**	.pem file used instead of .ppk	Convert the key with PuTTYgen (Step 13) and select the .ppk file.
PuTTY: **Server refused our key**	Wrong login name or wrong key	Log in as ec2-user and use the key created for this instance.
Cannot connect after restarting the instance	Public IP and DNS changed	Copy the new address into your command, MobaXterm session or PuTTY saved session.

Fix key permissions on Windows (CMD)

```
icacls "my-all-key.pem" /inheritance:r
icacls "my-all-key.pem" /grant:r "%username%:R"
```

The first command removes inherited permissions; the second gives only your account read access. Run both in the folder that contains the key, then retry the ssh command.

WELL DONE

You can now reach your Linux server with a key pair from CMD, Git Bash, MobaXterm and PuTTY. Next steps: install a web server with `sudo yum install httpd -y` or upload files using the MobaXterm SFTP panel.